```
irb(main)> [1, 2, 3].map(&iterate)
=> [2, 4, 6]

irb(main)> iterate.(2)
4
```

Ruby Procs can be assigned to variables and passed around or can be defined inline and used only by a single method. This is exactly how anonymous functions in Elixir work.

Anonymous functions can be defined across multiple lines (similar to Ruby Procs) or can be defined on a single line. You must put a period character between the variable name and the function arguments when you invoke an anonymous function, like my_function.(my_argument).

A major similarity between Ruby and Elixir is that the last statement executed by a function is returned. There is no explicit return syntax in Elixir. Instead, you use conditionals and pattern matching to structure your functions so the intended value is returned. This becomes fairly natural as you get the hang of Elixir.

Next, let's look at how functions can be overloaded.

Overloading Your Functions

An overloaded function is a function that has multiple entry points based on the input arguments. Some languages limit function overloading to the number of arguments—my_func/0 and my_func/1 would be considered overloaded. But Elixir goes beyond this.

In Elixir, you can overload the same function based on the argument data itself. So, my_func("hello") could have a different function body than my_func("hola").

Let's write an overloaded function in Elixir. Create lib/examples/modules/two.ex and add the following code:

elixir_examples/lib/examples/modules/two.ex

```
defmodule Examples.Modules.Two do
  def hello do
    "Hello"
  end

  # This is a reference to Erlang: The Movie
  def hello("Mike") do
    "Hello Mike, Hello Joe"
  end

  def hello(name) do
    "Hello #{name}"
  end
```